

MODULE 17

Ensemble Methods and Boosting

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Ensemble Methods and Boosting

What are Ensemble Methods?

Ensemble methods combine multiple models to improve performance.

Why they work: individual models make different errors;
combining reduces overall error.

Types:

Bagging: train models on random subsets (Random Forest)

Boosting: train sequentially, fix previous errors

Stacking: train meta-model on model predictions

Voting: majority vote or average predictions

Random Forest

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
import matplotlib.pyplot as plt

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    max_features='sqrt',
    min_samples_leaf=5,
    bootstrap=True,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train)

# Feature importance
importances = pd.Series(rf.feature_importances_, index=feature_names)
importances.sort_values().plot(kind='barh')
plt.title('Feature Importance')
plt.show()
```

Gradient Boosting and XGBoost

```
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# XGBoost
xgb = XGBClassifier(
    n_estimators=500, learning_rate=0.05,
    max_depth=6, subsample=0.8,
```

```

colsample_bytree=0.8, use_label_encoder=False,
eval_metric='logloss', random_state=42
)
xgb.fit(X_train, y_train, eval_set=[(X_val, y_val)], early_stopping_rounds=20)

# LightGBM (faster, memory efficient)
lgbm = LGBMClassifier(n_estimators=500, learning_rate=0.05, num_leaves=31)
lgbm.fit(X_train, y_train)

```

Stacking and Voting

```

from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Voting (soft = probability average)
voting = VotingClassifier(
    estimators=[('rf', RandomForestClassifier()), ('gb', GradientBoostingClassifier())],
    voting='soft'
)
voting.fit(X_train, y_train)

# Stacking
estimators = [('rf', RandomForestClassifier()), ('gb', GradientBoostingClassifier())]
stacking = StackingClassifier(
    estimators=estimators, final_estimator=LogisticRegression(), cv=5
)
stacking.fit(X_train, y_train)

```

Optuna for Hyperparameter Optimization

```

import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    n_estimators = trial.suggest_int('n_estimators', 100, 500)
    max_depth = trial.suggest_int('max_depth', 3, 15)
    min_samples = trial.suggest_int('min_samples_split', 2, 20)
    model = RandomForestClassifier(
        n_estimators=n_estimators, max_depth=max_depth, min_samples_split=min_samples
    )
    return cross_val_score(model, X, y, cv=5).mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
print('Best params:', study.best_params)

```